جلسهٔ ۱۵ — بازشناسیِ دنباله و Sequence OCR & Recognition (OCR)

جزوهٔ درس — هفتهٔ پانزدهم

فهرست مطالب

۱ نگاهِ کلی به این جلسه ۱

۱.۱ یک تشبیهِ ساده ۱

۲ از یک رقم به یک رشته ۲

۱.۲ رویکردِ اول: برش، سپس طبقه‌بندی ۲

۲.۲ ضربِ خطاها: تلهٔ استقلال ۲

۳.۲ چرا برش سخت است ۲

۳ رویکردِ سرتاسری: ستون‌های ویژگی ۳

۱.۳ مسئلهٔ ترازبندی ۳

۴ معماریِ CRNN ۴

۱.۴ مرحلهٔ ۱: CNN و ردیابیِ شکلِ تنسور ۴

۲.۴ مرحلهٔ ۲: map-to-sequence ۵

۳.۴ مرحلهٔ ۳: BiLSTM بافت می‌افزاید ۵

۵ آموزش بدونِ ترازبندی: CTC ۵

۱.۵ نمادِ بلنک، دقیق ۵

۲.۵ مسیرهای بسیار، یک رشته ۶

۳.۵ ایدهٔ CTC: جمع روی همهٔ هم‌ترازی‌ها ۶

۴.۵ CTC با دست: مثالِ کوچک . . . ۶
۵.۵ چرا این جمع محاسبه‌پذیر است . . . ۷

۶ کدگشایی و ارزیابی ۷

۱.۶ کدگشاییِ حریصانه (best-path) . . . ۷
۲.۶ جستجوی پرتو (beam search) . . . ۷
۳.۶ ارزیابی: نه فقط درست/غلط . . . ۸
۴.۶ فاصلهٔ ویرایش با یک مثال . . . ۸

۷ کدِ CTC + CRNN در PyTorch ۹

۸ تمرینِ کلاسی و پاسخ ۱۰

۹ فهرستِ عیب‌یابیِ عملی ۱۱

۱۰ جمع‌بندی ۱۱

۱ نگاهِ کلی به این جلسه

تا اینجا سفرِ DocScan را قدم‌به‌قدم ساختیم: در هفتهٔ ۱۱ تصویرِ کنتور را با هوموگرافی صاف کردیم، در هفتهٔ ۱۳ یک CNN ساختیم که یک رقم را با دقتِ ۹۷٪ می‌خواند، و در هفتهٔ ۱۴ با تشخیصِ شیء پنجرهٔ عددِ کنتور را پیدا کردیم. اما کنتورِ آب یک رقم ندارد؛ یک **رشته** از ارقام دارد، مثلِ 018426. کارِ این هفته خواندنِ کلِ آن رشته در یک گام است — نه بریدنِ آن به تک‌رقم‌ها و خواندنِ جداگانه.

> **یادداشت**
>
> دستاوردِ واقعیِ این هفته: یک شبکهٔ واحد به نامِ **CRNN** که کلِ نوارِ کنتور را می‌گیرد و کلِ عدد را بیرون می‌دهد، بدونِ هیچ مرحلهٔ «قیچی‌کردنِ» ارقام. این آخرین قطعهٔ الگوریتمیِ خطِ لولهٔ DocScan پیش از فینالِ درس است.

واحدِ پیش‌بینیِ ما در این هفته عوض می‌شود: از یک کلاسِ منفرد (هفتهٔ ۱۳) به یک دنبالهٔ مرتب از کلاس‌ها. این تغییرِ به‌ظاهر کوچک، سه پرسشِ تازه پیش می‌آورد که کلِ جلسه دربارهٔ آن‌هاست: (۱) چرا نباید رشته را ببریم؟ (۲) چگونه شبکه‌ای بسازیم که کلِ رشته را بخواند؟ (۳) چگونه بدونِ دانستنِ «کدام ستون به کدام رقم مربوط است» آن را آموزش دهیم و ارزیابی کنیم؟

۱.۱ یک تشبیهِ ساده

تصور کنید کسی می‌خواهد یک جملهٔ دست‌نویس را بخواند. یک راه این است که ابتدا با خط‌کش جمله را به حرف‌های جدا ببرد و بعد هر حرف را بخواند. اما حرف‌ها اغلب به هم چسبیده‌اند و مرزِ دقیقشان معلوم نیست؛ اگر برش را غلط بزنیم، حتی خوانندهٔ عالیِ تک‌حرف هم چرت‌وپرت می‌خواند. راهِ طبیعی‌ترِ انسان این است که چشم را روی جمله بلغزاند و هم‌زمان از بافتِ اطراف کمک بگیرد: حرفی که به‌تنهایی مبهم است، کنارِ همسایه‌اش روشن می‌شود. رویکردِ سرتاسریِ امروز دقیقاً همین کار را می‌کند.

۲ از یک رقم به یک رشته

۱.۲ رویکردِ اول: برش، سپس طبقه‌بندی

بدیهی‌ترین ایده این است که از دستاوردِ هفتهٔ ۱۳ استفاده کنیم: نوار را به تک‌رقم‌ها ببریم و CNN آماده را روی هر کدام اجرا کنیم، سپس نتایج را کنارِ هم بچینیم. این «تقسیم و غلبه» در ظاهر تمیز است، اما بر یک فرضِ شکننده استوار است: اینکه بتوانیم بینِ ارقام تمیز برش بزنیم.

۲.۲ ضربِ خطاها: تلهٔ استقلال

فرض کنیم برش‌گر بی‌نقص باشد و هر رقم با دقتِ a خوانده شود. یک عددِ L رقمی فقط وقتی کاملاً درست است که همهٔ ارقامش درست باشند:

ریاضیات

$$P(\text{کلِ رشته درست}) = a^L$$

برای $a = ۰٫۹۷$ (همان دقتِ CNN هفتهٔ ۱۳) و $L = ۶$:

$$۰٫۹۷^۶ \approx ۰٫۸۳۳$$

یعنی خوانندهٔ ۹۷٪-ی ما به یک خوانندهٔ رشته ۸۳٪-ی تبدیل می‌شود.

جدولِ زیر این افت را نشان می‌دهد:

دقتِ هر رقم a	$L=۱$	$L=۴$	$L=۶$	$L=۸$
۹۷٫۰	۹۷٫۰	۸۸۵٫۰	۸۳۳٫۰	۷۸۳٫۰
۹۹٫۰	۹۹٫۰	۹۶۱٫۰	۹۴۱٫۰	۹۲۳٫۰

نکتهٔ کلیدی

این محاسبه خوش‌بینانه است، چون فرض می‌کند برش هرگز خطا نمی‌کند. در عمل، خطای برش هم به خطای طبقه‌بندی اضافه می‌شود و افت را تندتر می‌کند. درسِ کلیدی: هرگز دقتِ تک‌رقم را به‌جای دقتِ خواندنِ کل گزارش نکنید.

۳.۲ چرا برش سخت است

چند دلیلِ فیزیکیِ واقعی که چرا نمی‌توان روی نوارِ کنتور تمیز برش زد:

- **ارقامِ چسبیده:** گاهی بینِ دو رقم اصلاً فاصله‌ای نیست تا روی آن برش بزنیم.
- **چرخشِ چرخ‌دنده‌ای (odometer roll-over):** در کنتورهای مکانیکی، وقتی یک چرخ در حالِ چرخش است، پنجره نیمی از یک رقم و نیمی از رقمِ بعدی را نشان می‌دهد. کدام کلاس؟
- **فاصلهٔ متغیر، تابش، کجی و سایه:** فاصله‌ها ظاهر و ناپدید می‌شوند؛ درخششِ نور می‌تواند فاصله‌ای جعلی بسازد یا فاصلهٔ واقعی را پاک کند.
- **تعدادِ نامعلومِ ارقام:** پیش از خواندن باید تصمیم بگیرید که اصلاً چند رقم وجود دارد — که خودش یک مسئلهٔ جداست.

هشدار / دام

برش‌گر (segmenter) حلقهٔ ضعیفِ زنجیر است: اگر برش غلط باشد، حتی یک طبقه‌بندِ بی‌نقص هم آشغال می‌خواند. در سیستم‌های واقعیِ کنتورخوانی، بیشترِ خطاها از همین‌جا می‌آیند، نه از طبقه‌بندی.

۳ رویکردِ سرتاسری: ستون‌های ویژگی

به‌جای برش، نوار را به‌صورتِ یک دنباله می‌خوانیم. ایدهٔ محوری: CNN را نگه می‌داریم، اما **محورِ عرض را از بین نمی‌بریم**. در هفتهٔ ۱۳، pooling هم ارتفاع و هم عرضِ تصویر را کوچک می‌کرد تا در نهایت یک بردارِ واحد بماند. اینجا برعکس عمل می‌کنیم: ارتفاع را تا ۱ کوچک می‌کنیم اما عرض را (تقریباً) دست‌نخورده نگه می‌داریم.

نتیجه یک **دنباله از ستون‌های ویژگی** است: به‌جای یک بردار، دنباله‌ای به طولِ T که هر عضوش یک بردارِ ویژگیِ C-بُعدی است. هر ستون به یک برشِ عمودیِ نوار «نگاه می‌کند» (میدانِ دیدش). نکتهٔ ظریف: این ستون‌ها هم‌پوشان‌اند و هیچ‌کدام مجبور نیستند دقیقاً یک رقم را در بر بگیرند — یک رقم ممکن است روی چند ستون پخش شود و یک رقمِ باریک با همسایه‌اش یک ستون را قسمت کند. این «بریدنِ نرم و هم‌پوشان» جایگزینِ برشِ سختِ رویکردِ اول است.

۱.۳ مسئلهٔ ترازبندی

اما حالا یک مشکلِ تازه داریم. CNN مثلاً $T = ۱۲$ ستون می‌دهد، ولی برچسبِ واقعی $L = ۶$ کاراکتر دارد. پس $T \neq L$، و ما نمی‌دانیم کدام ستون به کدام رقم مربوط است — هیچ علامتِ برشی وجود ندارد. رقمی که روی سه ستون پخش شده، باعث می‌شود شبکه بگوید «۴، ۴، ۴» — سه پیش‌بینی برای یک رقم.

نکتهٔ کلیدی

این «مسئلهٔ ترازبندی» (alignment problem) قلبِ کارِ امروز است: پیش‌بینیِ هر ستون را داریم، اما برچسبی کوتاه‌تر و بدونِ تراز داریم. راه‌حل، الگوریتمی به نامِ **CTC** است که در ادامه می‌آید.

یک اشارهٔ اولیه به راه‌حل (بدونِ ریاضی): بگذارید شبکه یک نمادِ ویژه به نامِ **بلنک**، (blank) با نمادِ خط‌تیره) هم بیرون بدهد که یعنی «اینجا کاراکترِ تازه‌ای نیست». سپس خروجیِ خامِ هر ستون را با دو قاعده، به‌ترتیب، فروکاست می‌کنیم: (۱) تکرارهای مجاور را ادغام کن، سپس (۲) بلنک‌ها را حذف کن. مثال:

ریاضیات

$$\text{خام: } ۴۴-۲۲۲-۶ \xrightarrow{\text{ادغامِ تکرار}} ۴-۲-۶ \xrightarrow{\text{حذفِ بلنک}} ۴۲۶$$

بلنک همان چیزی است که می‌گذارد یک رقمِ تکرارِ واقعی زنده بماند: 77 به 7 فرومی‌کاهد، اما 7 – 7 به 77 — بلنکِ میانی مانعِ ادغام می‌شود.

۴ معماریِ CRNN

CRNN یعنی شبکهٔ عصبیِ کانولوشنیِ بازگشتی (Convolutional Recurrent Neural Network)، ستون‌فقراتِ استانداردِ OCR. سه مرحله دارد:

مرحله	کارش	چرا
۱. CNN	نوار → ستون‌های ویژگی	استخراجِ ویژگی، حفظِ محورِ عرض
۲. map-to-sequence	کوچک‌کردنِ ارتفاع تا ۱	تبدیل به دنباله
۳. BiLSTM	بافت در هر دو جهت	بستنِ ابهام با همسایه‌ها
رونویسی (CTC)	امتیازِ ستون‌ها → رشته	خروجیِ نهایی

۱.۴ مرحلهٔ ۱: CNN و ردیابیِ شکلِ تنسور

بیایید شکلِ تنسور را در هر گام دنبال کنیم (برای نوارِ ۶ رقمی، هر رقم ۸×۸):

گام	خروجی $(C \times H \times W)$	توضیح
ورودی	۱ × ۸ × ۴۸	نوارِ ۶ رقمی
Conv۱ + pool ۲×۲	۱۶ × ۴ × ۲۴	کوچک‌شدنِ هر دو محور
Conv۲ + pool ۲×۲	۳۲ × ۲ × ۱۲	رشدِ عمق
Conv۳ + pool ۲×۱	۳۲ × ۱ × ۱۲	ارتفاع → ۱، عرض حفظ
فشردنِ ارتفاع	۳۲ × ۱۲	دنباله‌ٔ طولِ ۱۲

نکتهٔ کلیدی همان pooling آخر است: ۱ × ۲ یعنی ارتفاع را نصف کن اما عرض را دست نزن. نتیجه $T = ۱۲$ ستون است که هر کدام یک بردارِ ۳۲-بُعدی است.

۲.۴ مرحلهٔ ۲: map-to-sequence

تنسورِ CNN به شکلِ $(C, ۱, T)$ است. ارتفاعِ ۱ را فشرده می‌کنیم و سپس محورها را جابه‌جا (permute) می‌کنیم تا به (T, C) برسیم:

ریاضیات

$$(C, ۱, T) \xrightarrow{\text{squeeze}} (C, T) \xrightarrow{\text{permute}} (T, C)$$

حالا داده به‌معنای واقعی یک دنباله است: «زمان» = موقعیتِ افقی، «ویژگی» = کانال‌ها. ستونِ t آنچه CNN در برشی عمودی حولِ $x \approx ۴t$ دیده را حمل می‌کند.

۳.۴ مرحلهٔ ۳: BiLSTM بافت می‌افزاید

هر ستون واردِ یک **LSTM** می‌شود، واحدی بازگشتی که حافظه‌ای را در طولِ دنباله حمل می‌کند. دوسویه (bidirectional) یعنی یک گذر از چپ به راست و یک گذر از راست به چپ، سپس الحاق؛ حالا هر ستون هم می‌داند پیش از آن چه آمده و هم پس از آن. یک لایهٔ خطی، حالتِ پنهانِ هر ستون را به امتیازِ کلاس نگاشت می‌کند: ۱۰ رقم به‌علاوهٔ ۱ بلنک، یعنی ۱۱ کلاس.

یادداشت

همین بافت است که می‌گذارد یک رقمِ نیم‌چرخیده هم خوانده شود: همسایه‌هایش محدود می‌کنند که چه می‌تواند باشد. CNN می‌گوید «این برش چه شکلی است»، و RNN می‌گوید «با توجه به برش‌های اطراف، چه چیزی معنا می‌دهد».

خروجیِ نهاییِ CRNN یک ماتریسِ $T \times ۱۱$ است: برای هر ستون یک توزیعِ احتمال روی ۱۱ کلاس. حالا دو پرسش می‌ماند: چگونه این را آموزش دهیم، و چگونه به یک رشته تبدیلش کنیم؟

۵ آموزش بدونِ ترازبندی CTC:

۱.۵ نمادِ بلنک، دقیق

یک کلاسِ اضافه اضافه می‌کنیم، **بلنک**، به‌معنای «در این ستون کاراکترِ تازه‌ای منتشر نکن». یک خروجیِ خامِ هر-ستون را یک مسیر (path) می‌نامیم، و آن را با دو قاعده به متن فرومی‌کاهیم: اول برچسب‌های مجاورِ برابر را در یکی ادغام کن، سپس همهٔ بلنک‌ها را حذف کن. مثالِ مهم:

ریاضیات

$$\text{۷ ۷ ۷} \;\to\; \text{۷} \qquad \text{اما} \qquad \text{۷ - ۷} \;\to\; \text{۷۷}$$

۲.۵ مسیرهای بسیار، یک رشته

هدف 42 با $T = ۴$ ستون و الفبای $\{-, ۲, ۴\}$. همهٔ این مسیرها به 42 فرومی‌کاهند:

مسیر	فروکاست	مسیر	فروکاست
4 4 2 2	42	- 4 2 -	42
4 2 2 2	42	4 - 2 2	42
4 4 4 2	42	4 2 - -	42

مسیرهای متمایزِ فراوانی به یک رشتهٔ موردِنظرِ ما مربوط‌اند. ما هرگز انتخاب نکردیم کدام مسیر «درست» است — و لازم هم نیست.

۳.۵ ایدهٔ CTC: جمع روی همهٔ هم‌ترازی‌ها

ریاضیات

احتمالِ یک مسیر = حاصل‌ضربِ احتمال‌های هر ستون (ستون‌ها با شرطِ ویژگی‌ها مستقل فرض می‌شوند). احتمالِ یک برچسب = **جمعِ** احتمالِ همهٔ مسیرهایی که به آن فرومی‌کاهند:

$$P(\ell \mid x) = \sum_{\pi \,:\, \mathcal{B}(\pi) = \ell} \prod_{t=۱}^{T} p_t(\pi_t), \qquad \text{زیان} = -\log P(\ell \mid x)$$

که $\mathcal{B}$ همان قاعدهٔ فروکاست است. آموزش این احتمال را بالا می‌برد — برای همهٔ هم‌ترازی‌های معتبر هم‌زمان.

نکتهٔ کلیدی

هیچ علامتِ برشی لازم نیست: CTC می‌گذارد شبکه خودش ترازبندی را کشف کند، با پاداش‌دادن به همهٔ ترازبندی‌های ممکن. این تفاوتِ بنیادیِ CTC با آموزشِ نظارتیِ معمولی است، که در آن باید برای هر خروجی یک برچسبِ هم‌تراز داشته باشیم.

۴.۵ CTC با دست: مثالِ کوچک

الفبای $\{a, \text{-}\}$، $T = ۲$، هدف a. فرض کنیم احتمال‌های هر ستون این‌ها باشند:

	$p(a)$	$p(\text{-})$
ستونِ ۱	۰٫۷	۰٫۳
ستونِ ۲	۰٫۶	۰٫۴

مسیرهایی که به a فرومی‌کاهند: a a، a -، - a، a a.

ریاضیات

$$P(\text{a}) = \underbrace{۰٫۷ \times ۰٫۶}_{aa} + \underbrace{۰٫۷ \times ۰٫۴}_{a\text{-}} + \underbrace{۰٫۳ \times ۰٫۶}_{\text{-}a} = ۰٫۴۲ + ۰٫۲۸ + ۰٫۱۸ = ۰٫۸۸$$

زیان $-\log ۰٫۸۸ \approx ۰٫۱۲۸$. تنها مسیرِ مستثنا - - است که به رشتهٔ تهی فرومی‌کاهد.

۵.۵ چرا این جمع محاسبه‌پذیر است

تعدادِ مسیرها به‌صورتِ نمایی با T رشد می‌کند و برای نوارهای واقعی نمی‌توان همه را شمرد. اما قاعدهٔ فروکاست ساختار دارد: یک گذرِ **برنامه‌ریزیِ پویا** (الگوریتمِ روبه‌جلوی CTC) همهٔ هم‌ترازی‌ها را در زمانِ $O(T \cdot L)$ جمع می‌زند. شما هرگز این را خودتان پیاده نمی‌کنید؛ دستورِ `nn.CTCLoss` آن را به‌صورتِ مشتق‌پذیر انجام می‌دهد. (همان ترفندِ جمع‌زدنِ مسیرها در یک شبکهٔ مشبک: به‌جای پیمودنِ دوبارهٔ هر مسیر، جمع‌های جزئی را دوباره استفاده می‌کنیم.)

۶ کدگشایی و ارزیابی

۱.۶ کدگشاییِ حریصانه (best-path)

در زمانِ استنتاج یک رشته می‌خواهیم. ساده‌ترین راه: در هر ستون کلاسِ بیشینه (argmax) را بگیر، سپس قاعدهٔ فروکاست را اعمال کن. این **کدگشاییِ حریصانه** است: سریع و معمولاً خوب، اما ستون‌به‌ستون تصمیم می‌گیرد و ممکن است بهترین رشته را

از دست بدهد.

ریاضیات

۴۲۶ → حذفِ بلنک → ۶ - ۲ - ۴ → ادغام → ۶۶ - ۲۲ - ۴۴ :argmax

۲.۶ جستجوی پرتو (beam search)

کدگشاییِ حریصانه می‌تواند غلط باشد، چون بهترین مسیر همیشه بهترین رشته نیست (چند مسیر با هم به یک رشته رأی می‌دهند). **جستجوی پرتو** بهترین k رشتهٔ جزئی را نگه می‌دارد و احتمالِ جمع‌شدهٔ هر رشته را انباشته می‌کند؛ به هدفِ CTC وفادارتر است اما پرهزینه‌تر. یک مدلِ زبانی را هم می‌توان همین‌جا تزریق کرد.

نکتهٔ کلیدی

قاعدهٔ سرانگشتی: با حریصانه شروع کنید؛ فقط اگر دقتِ کافی نگرفتید سراغِ جستجوی پرتو بروید.

۳.۶ ارزیابی: نه فقط درست/غلط

سه سنجهٔ مهم:

- **دقتِ دنباله (sequence accuracy):** کسری از خواندن‌ها که دقیقاً درست‌اند. سخت‌گیر — یک رقمِ غلط، کلِ رشته را ردّ می‌کند.
- **نرخِ خطای کاراکتر (CER):** چند ویرایشِ تک‌کاراکتری برای اصلاحِ خواندن لازم است، به‌ازای هر کاراکتر.
- **فاصلهٔ ویرایش (edit distance / Levenshtein):** کمترین تعدادِ درج، حذف و جایگزینیِ تک‌کاراکتری برای تبدیلِ یک رشته به دیگری.

CER سنجهٔ منصف‌تری است: خواندنی که فقط یک رقم اشتباه دارد را صفر نمی‌دهد، بلکه به آن اعتبارِ نسبی می‌دهد.

۴.۶ فاصلهٔ ویرایش با یک مثال

ریاضیات

بازگشتیِ فاصلهٔ ویرایش:

$$D[i,j] = \min \begin{cases} D[i-۱, j] + ۱ & \text{(حذف)} \\ D[i, j-۱] + ۱ & \text{(درج)} \\ D[i-۱, j-۱] + \mathbb{1}[a_i \neq b_j] & \text{(تطبیق/جایگزینی)} \end{cases}$$

مرجع 018426، پیش‌بینی 018326: تنها یک جایگزینی (۳ → ۴)، پس فاصلهٔ ویرایش $= ۱$، و

$$\text{دقتِ دنباله برای این نمونه} = ۰ \qquad \text{CER} = \tfrac{۱}{۶} \approx ۰٫۱۶۷,$$

هشدار / دام

اگر خواننده یک رقم را بیندازد (01826) یا یک رقمِ اضافه توهم کند (0184426)، مقایسهٔ کاراکتربه‌کاراکتر در موقعیت‌های ثابت خراب می‌شود: همه‌چیز پس از لغزش غلط به‌نظر می‌رسد. فاصلهٔ ویرایش دقیقاً برای همین ساخته شده و درج/حذف را درست مدیریت می‌کند.

۷ کدِ CRNN + CTC در PyTorch

هستهٔ شبکه دقیقاً همان سه مرحله است:

```
class CRNN(nn.Module):
    def __init__(self, n_classes=11):        # 10 digits + blank
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d((2, 1)))            # height -> 1, width kept
        self.rnn = nn.LSTM(32, 64, bidirectional=True)
        self.fc  = nn.Linear(128, n_classes)
    def forward(self, x):                    # x: (B, 1, 8, W)
        f = self.cnn(x)                      # (B, 32, 1, T)
```

```
        f = f.squeeze(2).permute(2, 0, 1)     # map-to-seq: (T, B, 32)
        y, _ = self.rnn(f)                    # (T, B, 128)
        return self.fc(y).log_softmax(2)      # (T, B, n_classes)
```

و آموزش با زیانِ CTC:

```
ctc = nn.CTCLoss(blank=10, zero_infinity=True)
logp = net(xb)                                # (T, B, 11) log-probs
loss = ctc(logp, targets, input_lengths, target_lengths)
loss.backward(); opt.step()
```

در اینجا `input_lengths` تعدادِ ستون‌های واقعیِ هر نمونه (T) و `target_lengths` طولِ برچسبِ هر نمونه (L) است. یک فراخوانی روی همهٔ هم‌ترازی‌ها جمع می‌زند.

هشدار / دام

شایع‌ترین باگِ CTC: اشتباه‌گرفتنِ اندیسِ `blank`، یا نگه‌نداشتنِ ترتیبِ (T, B, C)، یا فراموش‌کردنِ `log_softmax`. در این حالت‌ها زیان به `inf` یا `nan` می‌رود. اگر $T < L$ باشد هم هیچ هم‌ترازیِ معتبری وجود ندارد و آموزش شکست می‌خورد.

کدگشاییِ حریصانه فقط شش خط است — همان قاعدهٔ فروکاست:

```
def greedy_decode(logp, blank=10):            # logp: (T, n_classes)
    path = logp.argmax(1).tolist()
    out, prev = [], None
    for c in path:
        if c != prev and c != blank:          # merge repeats, drop blank
            out.append(c)
        prev = c
    return out
```

یادداشت

دمو `crnn_ctc.py` نوارهای مصنوعیِ کنتور می‌سازد (به‌هم‌چسباندنِ تصادفیِ ۳ تا ۶ رقم از ارقامِ هفتهٔ ۱۳)، CRNN را با CTC روی CPU آموزش می‌دهد، و نوارهای آزمون را حریصانه کدگشایی می‌کند. در اجرای نمونهٔ ما ظرفِ ۲۵ epoch به دقتِ دنباله حدودِ ۹۹٪ و CER نزدیکِ صفر رسید. این یک دمو آموزشی است، نه یک کنتورخوانِ محصولی: نشان می‌دهد CER کاهش می‌یابد و قاعدهٔ فروکاست کار می‌کند.

۸ تمرینِ کلاسی و پاسخ

تمرین

۱) مسیرِ `- 5 - 5 5 -` را با قاعدهٔ CTC فروکاست کنید. در رشتهٔ نهایی چند تا ۵ می‌ماند و چرا؟

۲) الفبای $\{a, -\}$، $T = ۲$، هدف `aa`. کدام تک‌مسیر `aa` را می‌سازد و اگر $p_t(a) = ۰/۵$ باشد احتمالش چقدر است؟

۳) مرجع `7013`، پیش‌بینی `713`. فاصلهٔ ویرایش و CER چند است؟

۴) زیانِ CTC بلافاصله `inf` می‌شود. دو علتِ محتمل بگویید.

پاسخ‌ها: (۱) در `- 5 - 5 5 -`، ابتدا `5 5` مجاور به یک `5` ادغام می‌شود، اما `5`ِ بعدی با یک بلنک از آن جدا شده؛ پس دو تا ۵ می‌ماند (`55`). بلنکِ میانی مانعِ ادغام است. (۲) با $T = ۲$ تنها `a -` و `- a` به `aa` فرومی‌کاهد (هر بلنکی آن را کوتاه می‌کند)؛ احتمال $۰/۵ \times ۰/۵ = ۰/۲۵$. (۳) یک حذف (رقمِ ۰ افتاده): فاصلهٔ ویرایش $= ۱$، و $\text{CER} = ۱/۴ = ۰/۲۵$. (۴) اندیسِ `blank` غلط (یا لاجیت‌ها log-softmax نشده‌اند)، یا طولِ ورودی $T < L$ به‌طوری‌که هیچ هم‌ترازیِ معتبری وجود ندارد.

۹ فهرستِ عیب‌یابیِ عملی

وقتی خوانندهٔ رشتهٔ خودتان کار نمی‌کند، این فهرست را به‌ترتیب چک کنید:

- **زیان `nan`/`inf` شد؟** اندیسِ بلنک، ترتیبِ (T, B, C) و `log_softmax` را بررسی کنید؛ مطمئن شوید $T \geq L$ برای همهٔ نمونه‌ها.
- **زیان پایین اما خواندن خالی است؟** شبکه یاد گرفته همه‌جا بلنک بزند (یک کمینهٔ محلیِ رایج در آغازِ آموزش)؛ نرخِ یادگیری را تنظیم کنید یا کمی بیشتر صبر کنید.
- **ارقامِ تکراری گم می‌شوند؟** احتمالاً بلنک بینِ آن‌ها آموزش ندیده؛ داده‌ای با ارقامِ تکراریِ کافی لازم است.
- **دقتِ دنباله پایین اما CER پایین است؟** خواننده معمولاً نزدیک است اما یک لغزشِ کوچک دارد؛ این طبیعی است و دقیقاً چرایِ گزارشِ هر دو سنجه را نشان می‌دهد.
- **همه‌چیز درست به‌نظر می‌رسد اما جواب بد است؟** ترازنبودنِ تصویر و برچسب را چک کنید (همان درسِ هفتهٔ ۱۳).

هشدار / دام

پیش از آموزشِ بزرگ، شبکه را روی چند نمونه امتحان کنید و ببینید آیا می‌تواند همان‌ها را حفظ کند (به CER صفر برسد). اگر نتواند، باگی اساسی در شکلِ تنسور، اندیسِ بلنک یا طول‌ها دارید.

۱۰ جمع‌بندی

در این جلسه یاد گرفتیم که خواندنِ یک رشته با خواندنِ یک رقم فرق دارد: رویکردِ «برش سپس طبقه‌بندی» خطاها را ضرب می‌کند و روی ارقامِ چسبیده و چرخانِ کنتور می‌شکند. راه‌حلِ سرتاسری، CRNN است: CNN ستون‌های ویژگی می‌سازد (محورِ عرض را حفظ می‌کند)، map-to-sequence آن را به دنباله تبدیل می‌کند، BiLSTM بافت می‌افزاید، و لایهٔ رونویسی امتیازها را به رشته بدل می‌کند. زیانِ **CTC** با جمع‌زدن روی همهٔ هم‌ترازی‌ها می‌گذارد شبکه بدونِ علامتِ برش آموزش ببیند؛ کدگشاییِ حریصانه (یا جستجوی پرتو) خروجی را به رشته تبدیل می‌کند؛ و با فاصلهٔ ویرایش، CER و دقتِ دنباله آن را صادقانه می‌سنجیم. با این قطعه، خطِ لولهٔ DocScan — صاف‌کردن، تشخیص، و حالا **خواندن** — کامل می‌شود و آمادهٔ فینالِ درس است.